Exercise 1: Floating point numbers

Prognostic Modelling (pea700), WiSe 2026/27

Out: week 1 · Back: week 2 · 7 points

Covers unit 1 of the script. The preparation sets up the repository through which you hand in all exercises of the semester; see the rules at its end. The preparation and all three problems are worked on in the lab; what you do not finish there is homework.

Preparation: Git, GitLab and your working environment

All solutions of this semester and the final project report are handed in through the institute's GitLab server, `https://<gitlab-address>`. You will receive an invitation by e-mail during the first lab. GitLab stores your work in a **git repository**: a folder whose complete history of changes is recorded, so that every earlier state can be restored and every change can be traced. Version control is standard in scientific programming, and you will use it for every exercise.

This preparation carries no points, but without it you cannot hand in. You may use any editor. We recommend **Visual Studio Code** (VS Code), which combines an editor, a terminal, Jupyter notebooks and git in one window; see the box at the end of this section.

a) **Set up git.** Check that git is installed with `git --version` (on Windows, install *Git for Windows* from git-scm.com). Tell git who you are; this name appears in every commit you make:

```
git config --global user.name "First Last"
git config --global user.email "your.address@uni-bonn.de"
git config --global init.defaultBranch main
```

Create an SSH key, which lets you push without typing a password, and add the **public** key to GitLab under *Preferences → SSH Keys*:

```
ssh-keygen -t ed25519 -C "your.address@uni-bonn.de"
cat ~/.ssh/id_ed25519.pub        # copy this line into GitLab
ssh -T git@<gitlab-address>      # test: GitLab greets you by name
```

Never share the private key `~/.ssh/id_ed25519`.

b) **Get your repository.** Your personal repository has already been created in the course group `<course-group>`. It is named after the initial of your first name and your last name, for example `jsmith` for Jane Smith, and is visible only to you, the lecturer and the tutors. Accept the invitation from the e-mail, sign in, and copy the SSH address from the *Code* button of the project page. Clone the repository to your computer:

```
git clone git@<gitlab-address>:<course-group>/emustermann.git
cd emustermann
```

Give it the following structure: one folder per exercise sheet, one for the project, a `README.md` with your name and matriculation number, and a `.gitignore` that keeps generated files out of the repository.

```
jsmith/
    README.md
    .gitignore
    requirements.txt
    ex01/
    ex02/
    ...
    project/
```

with the `.gitignore`

```
.venv/
__pycache__/
.ipynb_checkpoints/
```

c) **Set up Python.** Create a virtual environment inside the repository, activate it, and install the packages of this course:

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install numpy scipy matplotlib jupyter
```

Write the four package names into `requirements.txt`, one per line, so that anybody can recreate the environment with `pip install -r requirements.txt`. The environment itself (`.venv/`) is not committed: it is large, and it only works on your computer.

d) **Commit and push.** Record the current state and send it to GitLab:

```
git status                       # what has changed?
git add README.md .gitignore requirements.txt
git commit -m "Set up repository for pea700"
git push
```

Check in the browser that the files have arrived. From now on every exercise follows the same cycle: work, `git add`, `git commit`, `git push`. If you work on more than one computer, start each session with `git pull`.

Rules for handing in

- Each sheet goes into its folder `exNN/`. Code as a notebook (`.ipynb`) or script (`.py`) that runs without changes in the environment from `requirements.txt`. Derivations as PDF (scan or typeset) or in the markdown cells of the notebook.
- What is pushed to GitLab before the deadline is graded. Push early and often; there is no penalty for many commits.
- Do not commit generated or large files (environments, data files, output folders).
- The final project report goes into `project/`.

Working with VS Code (optional)

1. Install VS Code from code.visualstudio.com and, in the *Extensions* view (`Ctrl+Shift+X`), the extensions **Python** and **Jupyter** (both by Microsoft).
2. *File* → *Open Folder* and choose your repository folder.
3. Select the environment: `Ctrl+Shift+P` → *Python: Select Interpreter* → `.venv`. Notebooks then use it as their kernel.
4. The *Source Control* view (`Ctrl+Shift+G`) shows changed files. Stage them with `+`, write a message, *Commit*, then *Sync Changes* to push and pull. The integrated terminal (`Ctrl+`` ` ``) accepts all commands above.

The Python you need for this sheet

Open a notebook or a Python file in your repository folder `ex01/` and try:

```
x = 0.1 + 0.2          # compute and store the result under the name x
print(x)               # show the value of x
print("x =", x)        # show a text and then the value
print(f"{x:.20f}")     # show x with 20 digits after the decimal point
print(x == 0.3)        # == compares two values: True or False

n = 0                  # a counter
while n < 3:           # repeat the indented lines as long as n < 3
    n = n + 1          # increase the counter by 1
    print("n =", n)    # prints n = 1, n = 2, n = 3
```

`=` stores a value, `==` compares two values. The lines that belong to the `while` loop are indented by four spaces; Python checks the condition before each pass and continues after the loop as soon as it is `False`. Everything else is introduced where it is needed.

Problem 1: Python as a calculator (in the lab, 2 points)

a) Before you run anything, write down what you **expect** for each line. Then run the lines and compare.

```
print(0.1 + 0.2 == 0.3)
print(f"{0.1:.20f}")
print(1 / 3 * 3 == 1)
print(1e16 + 1 - 1e16)
print(1e16 - 1e16 + 1)
print(10.0**308 * 10)
print(1e-320 / 1e10)
```

b) Explain each result in one sentence. Use the three facts about floating point numbers from the lecture: about 16 significant digits, a largest and a smallest number, and most decimal fractions are not exactly representable.

c) How should two floating point numbers `a` and `b` be compared instead of `a == b`? Write a comparison that gives `True` for `0.1 + 0.2` and `0.3`.

Solution

a) Each expression is computed and printed on its own line. The text in quotes is printed as it is; after the comma follows the value.

```
# 0.1 + 0.2 compared with 0.3
a = 0.1 + 0.2                       # compute the sum and store it under the name a
print("0.1 + 0.2 =", a)             # show the stored value
print("a == 0.3:", a == 0.3)        # == compares two values; the answer is True or
↪  False

# 0.1 with 20 digits after the decimal point
print("0.1 with 20 digits:", f"{0.1:.20f}")

# 1/3 times 3 compared with 1
b = 1 / 3 * 3
print("1 / 3 * 3 =", b)
print("b == 1:", b == 1)

# adding and subtracting a large number; 1e16 means 1 times 10 to the power 16
c = 1e16 + 1 - 1e16
print("1e16 + 1 - 1e16 =", c)
d = 1e16 - 1e16 + 1
print("1e16 - 1e16 + 1 =", d)

# a very large and a very small result; ** means "to the power of"
```

```
e = 10.0**308 * 10
print("10.0**308 * 10 =", e)
f = 1e-320 / 1e10
print("1e-320 / 1e10 =", f)
```

```
0.1 + 0.2 = 0.30000000000000004
a == 0.3: False
0.1 with 20 digits: 0.10000000000000000555
1 / 3 * 3 = 1.0
b == 1: True
1e16 + 1 - 1e16 = 0.0
1e16 - 1e16 + 1 = 1.0
10.0**308 * 10 = inf
1e-320 / 1e10 = 0.0
```

Some Python used here:

- `a = 0.1 + 0.2` computes the right hand side and stores the result under the name `a`. It is not an equation; it is an instruction.
- `a == 0.3` (two equal signs) *compares* and gives `True` or `False`.
- `f"{0.1:.20f}"` is a formatted text: the `f` before the quotes tells Python to insert the value in braces, and `.20f` means "with 20 digits after the decimal point".
- `1e16` is Python's way of writing 1×10^{16}, and `1e-320` is 1×10^{-320}.

b)

- `0.1 + 0.2 == 0.3` is `False`. None of the three numbers can be stored exactly in base 2. The sum is `0.30000000000000004`, which differs from the stored value of 0.3 in the last digit.
- `0.1` is stored as `0.10000000000000000555...`: the double closest to 0.1.
- `1 / 3 * 3 == 1` is `True`. Here the two rounding errors (in the division and in the multiplication) happen to cancel. Whether two floating point results are exactly equal is a matter of luck.
- `1e16 + 1 - 1e16` gives `0.0`. With about 16 significant digits, $10^{16} + 1$ cannot be distinguished from 10^{16}: the 1 is lost in the first addition.
- `1e16 - 1e16 + 1` gives `1.0`. The same numbers in a different order: now the large numbers cancel first, and the 1 survives. In floating point arithmetic the order of operations matters.
- `10.0**308 * 10` gives `inf` (infinity). The result is larger than the largest double, about 1.8×10^{308}: overflow.
- `1e-320 / 1e10` gives `0.0`. The result is smaller than the smallest double, about 5×10^{-324}: underflow.

c) Instead of asking "are `a` and `b` equal?" we ask "is their difference smaller than a tolerance?".

```
a = 0.1 + 0.2
b = 0.3
difference = abs(a - b)            # abs gives the absolute value, i.e. drops the
↪  sign
print("difference:", difference)

tolerance = 1e-12                  # what we accept as "equal"
print("equal within the tolerance:", difference < tolerance)
```

```
difference: 5.551115123125783e-17
equal within the tolerance: True
```

The difference is about 5.6×10^{-17}, far below the tolerance, so the comparison gives `True`. The tolerance must fit the size of the numbers: for numbers around 10^8, a difference of 10^{-12} is already below the accuracy of a double. Python's module `math` contains a function that chooses the tolerance relative to the size of the numbers:

```
import math                        # load the module math, which contains
↪  mathematical functions
print("math.isclose(a, b):", math.isclose(a, b))
```

```
math.isclose(a, b): True
```

Points: a) and b) 1, c) 1.

Problem 2: A time loop (in the lab, 2 points)

Every prognostic model has a time loop. Here is a simple one that should advance the time from 0 to 1 in steps of 0.1:

```
t = 0.0
dt = 0.1
steps = 0
while t < 1.0:
    t = t + dt
    steps = steps + 1
print(steps, t)
```

a) How many steps do you expect? Run the code. How many steps are made, and what is the final time? Explain.

b) Repeat with `dt = 0.25`. Why does the loop now behave as expected?

c) Rewrite the loop such that it always makes exactly the intended number of steps: compute the number of steps first, loop over an integer counter `n`, and compute the time from the counter as `t = n * dt`.

Solution

a) To see what happens, we print the time in every step. The `print` is the only change to the loop of the problem.

```
t = 0.0                             # start time
dt = 0.1                            # time step
steps = 0                           # counts the steps
while t < 1.0:                      # repeat the indented lines as long as t < 1
    t = t + dt                      # advance the time by one step
    steps = steps + 1               # count the step
    print("step", steps, "time", t)
print("number of steps:", steps)
```

```
step 1 time 0.1
step 2 time 0.2
step 3 time 0.30000000000000004
step 4 time 0.4
step 5 time 0.5
step 6 time 0.6
step 7 time 0.7
step 8 time 0.7999999999999999
step 9 time 0.8999999999999999
step 10 time 0.9999999999999999
step 11 time 1.0999999999999999
number of steps: 11
```

How the loop works: before each pass, Python checks the condition `t < 1.0`. If it is `True`, the indented lines are executed once more; if it is `False`, Python continues after the loop. The indentation (four spaces) tells Python which lines belong to the loop.

One expects 10 steps and a final time of 1. The loop makes **11** steps and ends at $t = 1.1$. The printout shows why: 0.1 is not exactly representable, and the small rounding errors add up from step to step. After ten steps t is 0.9999999999999999, which is still smaller than 1, so the loop runs once more. In a model this means one time step too many, or a forcing applied at the wrong time.

b)

```
t = 0.0
dt = 0.25                          # the only change
steps = 0
while t < 1.0:
    t = t + dt
    steps = steps + 1
    print("step", steps, "time", t)
print("number of steps:", steps)
```

```
step 1 time 0.25
step 2 time 0.5
step 3 time 0.75
step 4 time 1.0
number of steps: 4
```

Now the loop stops after 4 steps exactly at $t = 1$. The number 0.25 is 2^{-2}, a power of 2, and is stored exactly; all sums are therefore exact. Whether the loop of a) works thus depends on the time step, which makes such errors hard to find.

c) We first compute how many steps are needed. The number of steps is a whole number (an integer), and integers are stored exactly. The loop counts the steps, and the time is computed from the step number.

```
t_end = 1.0                        # end time
dt = 0.1                           # time step
n_steps = round(t_end / dt)        # number of steps, rounded to a whole number
print("number of steps:", n_steps)

n = 0                              # step counter, a whole number
while n < n_steps:
    n = n + 1
    t = n * dt                     # time computed from the counter, not accumulated
    print("step", n, "time", t)
```

```
number of steps: 10
step 1 time 0.1
step 2 time 0.2
step 3 time 0.30000000000000004
step 4 time 0.4
step 5 time 0.5
step 6 time 0.6000000000000001
step 7 time 0.7000000000000001
step 8 time 0.8
```

```
step 9 time 0.9
step 10 time 1.0
```

`round` rounds to the nearest whole number. It is needed because `t_end / dt` is itself a floating point number and could come out as `9.999999999999998` instead of 10.

The loop now always makes exactly 10 steps and ends exactly at $t = 1$. The times of some steps are still not exact (`0.6000000000000001` in step 6), because 6×0.1 is rounded. But this error is made once and does not add up from step to step as in a).

Experienced Python programmers write the same loop shorter as `for n in range(1, n_steps + 1):`, which lets `n` run through $1, 2, \ldots, 10$; it does exactly the same.

Points: a) 1, b) and c) 1.

Problem 3: Machine epsilon and cancellation (in the lab, 3 points)

a) Determine the machine epsilon experimentally: start with `eps = 1.0` and halve it as long as `1.0 + eps / 2 > 1.0`. Compare with the value from the lecture, $2^{-52} \approx 2.2 \times 10^{-16}$.

b) The equation $x^2 - 10^8 x + 1 = 0$ has the roots $x_{1,2} = \frac{1}{2}\left(10^8 \pm \sqrt{10^{16} - 4}\right)$. The smaller root is very close to 10^{-8}. Compute it with this formula (`math.sqrt` computes the square root). How many digits are correct, and why? Find a formula that gives the smaller root accurately. Hint: $x_1 x_2 = 1$.

Solution

a) We halve `eps` as long as adding half of it to 1 still changes 1, and count how often this happens.

```
eps = 1.0                          # start value
halvings = 0                       # counts the halvings
while 1.0 + eps / 2 > 1.0:         # does half of eps still change 1?
    eps = eps / 2                  # then halve eps
    halvings = halvings + 1
print("machine epsilon:", eps)
print("number of halvings:", halvings)
print("2 to the power -52:", 2.0**-52)
```

```
machine epsilon: 2.220446049250313e-16
number of halvings: 52
2 to the power -52: 2.220446049250313e-16
```

The loop stops after 52 halvings at $\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$: adding anything smaller than $\varepsilon/2$ to 1 leaves 1 unchanged. This is the distance between 1 and the next larger double, and it reflects the 52 bits that a double stores after the leading digit.

b) We compute the two roots step by step.

```
import math                          # the module math contains the square root
↪  math.sqrt

d = 1e16 - 4                         # the number under the square root
w = math.sqrt(d)                     # the square root
print("square root:", w)

x1 = (1e8 + w) / 2                   # larger root, + sign
x2_naive = (1e8 - w) / 2             # smaller root, - sign, the formula of the problem
print("larger root  x1:", x1)
print("smaller root x2, naive formula:", x2_naive)
```

```
square root: 99999999.99999999
larger root  x1: 100000000.0
smaller root x2, naive formula: 7.450580596923828e-09
```

Python writes very small numbers as `7.450580596923828e-09`, which means 7.45×10^{-9}. The correct value is very close to 10^{-8}, so the naive result is wrong already in the first digit. To check a root without knowing the exact value, we insert it into the equation; a correct root gives (almost) zero:

```
check_naive = x2_naive**2 - 1e8 * x2_naive + 1
print("x2 naive inserted into the equation:", check_naive)
```

```
x2 naive inserted into the equation: 0.2549419403076172
```

The remainder is 0.25 instead of 0: the naive root is wrong.

Why? The formula subtracts two numbers that are almost equal: 10^8 and $\sqrt{10^{16} - 4} \approx 10^8 - 2 \times 10^{-8}$. Both agree in their first 16 digits, so the difference consists of the rounding errors of the two numbers. Neighbouring doubles near 10^8 are about 1.5×10^{-8} apart, so the result cannot be more accurate than that, which is as large as the root itself. Not a single digit can be trusted.

The remedy. The larger root is computed with a sum, without cancellation, and is accurate. Since $x_1 x_2 = 1$, the smaller root is $x_2 = 1/x_1$:

```
x2_stable = 1 / x1
print("smaller root x2, stable formula:", x2_stable)

check_stable = x2_stable**2 - 1e8 * x2_stable + 1
print("x2 stable inserted into the equation:", check_stable)
```

```
smaller root x2, stable formula: 1e-08
x2 stable inserted into the equation: 1.1102230246251565e-16
```

Now the remainder is about 10^{-16}, zero within the accuracy of a double. Both formulas are mathematically identical; only the second avoids the subtraction of nearly equal numbers. The problem itself is well-conditioned; the first algorithm is unstable (unit 1 of the script, section on conditioning).

Points: a) 1, b) 2.